



南開大學
Nankai University

数据挖掘实验报告

学 号: 2213117
姓 名: 蔡 佳 良
年 级: 2022 级
学 院: 统计与数据科学学院
专 业: 统 计 学
完成日期: 2024 年 12 月 20 日

目录

1 第一次上机实验（提取图像的纹理特征）	3
1.1 实验要求	3
1.2 数据分析与处理	3
1.3 实验步骤与原理	3
1.3.1 灰度共生矩阵	3
1.3.2 局部二值模式	4
1.4 实验结果与分析	4
1.5 实验代码	5
2 第二次上机实验（垂直平分分类器）	10
2.1 实验要求	10
2.2 数据分析与处理	10
2.3 实验步骤与原理	10
2.4 实验结论与分析	10
2.5 实验代码	11
3 第三次上机实验（使用朴素贝叶斯方法进行预测）	15
3.1 实验要求	15
3.2 数据分析与处理	15
3.3 实验步骤与原理	15
3.4 实验结论与分析	17
3.5 实验代码	17
4 第四次上机实验（使用决策树进行预测）	21
4.1 实验要求	21
4.2 数据分析与处理	21
4.3 实验步骤与原理	21
4.4 实验结论与分析	21
4.5 实验代码	21

5 第五次上机实验（信用卡欺诈预测）	27
5.1 实验要求	27
5.2 数据分析与处理	27
5.3 实验步骤与原理	27
5.4 实验结论与分析	28
5.5 实验代码	28

第一次上机实验 (提取图像的纹理特征)

1.1 实验要求

- 给定若干张图像, 利用灰度共生矩阵特征或局部二值模式特征对这些图像进行特征提取;
- 图象是 $W * H * 3$ 的矩阵;
- 将最终提取到的特征通过 plot 的形式展示, 直观对比不同纹理提取到的特征;
- 使用 Python 编程实现。

1.2 数据分析与处理

将图像转化为灰度矩阵。

1.3 实验步骤与原理

1.3.1 灰度共生矩阵

灰度共生矩阵是从 $N \times N$ 的图像 $f(x, y)$ 的灰度为 i 的像素出发, 统计与距离为 $\delta = (dx^2 + dy^2)^{1/2}$, 灰度为 j 的像素同时出现的概率为, 数学表达式为:

$$P(i, j, \delta, \theta) = \{[(x, y), (x + dx, y + dy)] | f(x, y) = i, f(x + dx, y + dy) = j\},$$

分别考虑 $\theta = 0^\circ, 45^\circ, 90^\circ, 135^\circ$,

当 $\theta = 0^\circ$ 时, $dx = 1, dy = 0$;

当 $\theta = 45^\circ$ 时, $dx = 1, dy = -1$;

当 $\theta = 90^\circ$ 时, $dx = 0, dy = -1$;

当 $\theta = 135^\circ$ 时, $dx = -1, dy = -1$ 。

1.3.2 局部二值模式

LBP 算子定义为在 3×3 的窗口内，以窗口中心像素为阈值，将相邻的 8 个像素的灰度值与其进行比较，若周围像素值大于中心像素值，则该像素点的位置被标记为 1，否则为 0。这样， 3×3 邻域内的 8 个点经比较可产生 8 位二进制数（通常转换为十进制数即 LBP 码，共 256 种），即得到该窗口中心像素点的 LBP 值，并用这个值来反映该区域的纹理信息（例如亮点和暗点），数学表达式为：

$$LBP = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p,$$

其中，

- $s(x)$ 是符号函数：

$$s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases},$$

- P 是邻域像素数（通常为 8）。

1.4 实验结果与分析

通过两种方法下生成的特征图可以看出，两类图有明显不同的纹理特征。图 1.1 与图 1.2 为灰度共生矩阵下的图例；图 1.3 与图 1.4 为局部二值模式下的图例。全部图片请见[特征图](#)。

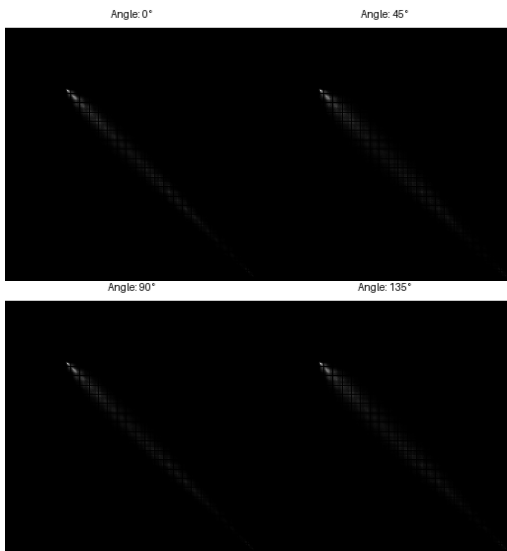


图 1.1: 云的灰度共生矩阵图

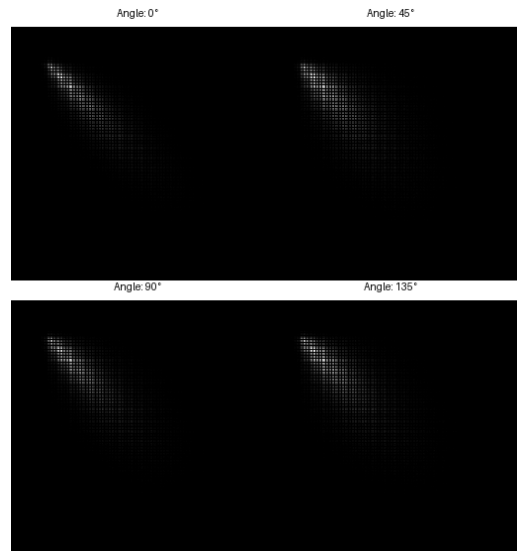


图 1.2: 森林的灰度共生矩阵图

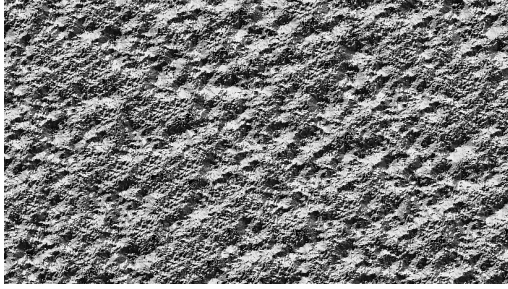


图 1.3: 云的局部二值模式图

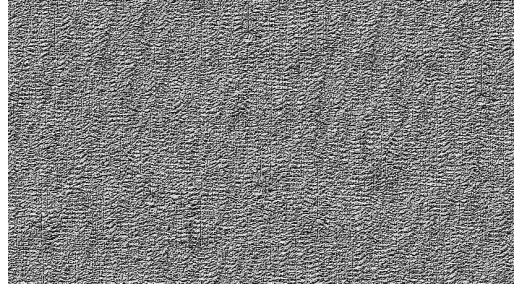


图 1.4: 森林的局部二值模式图

1.5 实验代码

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import matplotlib as mpl
5 from PIL import Image, ImageDraw, ImageFont
6
7 mpl.rcParams["font.sans-serif"] = ["SimHei"] # 中文
8 plt.rcParams["axes.unicode_minus"] = False # 正负号
9
10 # 更改当前工作目录为脚本所在目录
11 os.chdir(os.path.dirname(os.path.abspath(__file__)))
12
13 # 输入/输出目录
14 input_dirs = ["cloud", "forest"]
15 output_dirs = [
16     "output/glcm/cloud",
17     "output/glcm/forest",
18     "output/lbp/cloud",
19     "output/lbp/forest",
20 ]
21 for d in output_dirs:
22     os.makedirs(d, exist_ok=True)
23
24 # 用 basename 只保留文件名, 避免保存路径带目录前缀
25 def get_mat(names, in_dir, out_dir):
26     for i in names:
```

```
27         img = Image.open(os.path.join(in_dir, i))
28         img = img.convert("L") # 转为灰度图
29         img = np.array(img) # 转为ndarray
30         yield i, out_dir, img
31
32
33 # 计算灰度共生矩阵(gray-level co-occurrence matrix)
34 def get_glcm(img, angle=0, distance=1, gray_levels=256):
35     # 归一化到0-gray_levels灰度级
36     img = (img / (np.max(img) + 1e-5) * (gray_levels - 1)).
37           astype(int)
38     h, w = img.shape # 获取图像的尺寸
39     glcm = np.zeros((gray_levels, gray_levels), dtype=np.
40                     int64)
41
42     for i in range(h):
43         for j in range(w):
44             match angle:
45                 case 0:
46                     if j + distance < w:
47                         glcm[img[i, j], img[i, j + distance
48 ]] += 1
49                 case 90:
50                     if i + distance < h:
51                         glcm[img[i, j], img[i + distance, j
52 ]] += 1
53                 case 45:
54                     if i + distance < h and j + distance <
55 w:
56                         glcm[img[i, j], img[i + distance, j
57 + distance]] += 1
58                 case 135:
59                     if i + distance < h and j - distance >=
60 0:
61                         glcm[img[i, j], img[i + distance, j
62 - distance]] += 1
```

```
56     glcm = glcm / np.sum(glcm) # 归一化
57     return glcm
58
59
60 # 绘制灰度共生矩阵
61 def get_glcm_fig(names, in_dir, out_dir, angles=[0, 45, 90,
62     135]):
63     for i, out, img in get_mat(names, in_dir, out_dir):
64         glcms = [] # 存储各角度的 GLCM 图像
65         for angle in angles:
66             glcm = get_glcm(img, angle=angle)
67             glcm = (glcm / glcm.max() * 255).astype(np.
68                 uint8)
69             glcms.append((Image.fromarray(glcm), f"Angle: {
70                 angle}°")) # 保存图像和标题
71
72
73     # 计算单张 GLCM 图像的大小
74     width, height = glcms[0][0].size
75     title_height = 20 # 为标题预留的高度
76
77
78     # 2x2 布局, 标题增加额外空间
79     canvas = Image.new("L", (2 * width, 2 * (height +
80         title_height)), "white")
81
82
83     draw = ImageDraw.Draw(canvas)
84     font = ImageFont.load_default()
85
86     for idx, (glcm, title) in enumerate(glcms):
87         x_offset = (idx % 2) * width
88         y_offset = (idx // 2) * (height + title_height)
89
90         text_bbox = draw.textbbox((0, 0), title, font=
91             font)
92         text_width = text_bbox[2] - text_bbox[0]
93         text_x = x_offset + (width - text_width) // 2
94         draw.text((text_x, y_offset), title, fill="
95             black", font=font)
```

```
87
88         canvas.paste(glcm, (x_offset, y_offset +
            title_height))
89
90         canvas.save(os.path.join(out, i))
91
92
93 # 计算局部二值模式(local binary pattern)
94 def get_lbp(img):
95     h, w = img.shape # 获取图像的尺寸
96     lbp = np.zeros((h - 2, w - 2), dtype=np.int64) # 忽略边
        缘像素
97     for i in range(1, h - 1):
98         for j in range(1, w - 1):
99             center = img[i, j]
100             neighbors = [
101                 img[i, j - 1],
102                 img[i + 1, j - 1],
103                 img[i + 1, j],
104                 img[i + 1, j + 1],
105                 img[i, j + 1],
106                 img[i - 1, j + 1],
107                 img[i - 1, j],
108                 img[i - 1, j - 1],
109             ]
110             code = 0
111             for idx, neighbor in enumerate(neighbors):
112                 if neighbor > center:
113                     code += 2**idx
114             lbp[i - 1, j - 1] = code
115     return lbp
116
117
118 # 绘制局部二值模式
119 def get_lbp_fig(names, in_dir, out_dir):
120     for i, out, img in get_mat(names, in_dir, out_dir):
121         lbp = get_lbp(img)
```

```
122         Image.fromarray(lbp.astype(np.uint8)).save(os.path.  
            join(out, i))  
123  
124  
125 def main():  
126     for in_dir in input_dirs:  
127         names = os.listdir(in_dir)  
128         get_glcmm_fig(names, in_dir, f"output/glcmm/{in_dir}"  
            )  
129         get_lbp_fig(names, in_dir, f"output/lbp/{in_dir}")  
130  
131  
132 if __name__ == "__main__":  
133     main()
```

第二次上机实验（垂直平分分类器）

2.1 实验要求

- 由训练数据，训练一个垂直平分分类器；
- 对测试数据进行分类；
- 使用 Python 编程实现。

2.2 数据分析与处理

测试集与训练集均为已分类的二维数据。

2.3 实验步骤与原理

训练集中两类数据的均值分别为记为 m_1 和 m_2 ，则垂直平分形式的线性分类器为

$$g(\mathbf{x}) = (m_1)'(\mathbf{x} - (m_1 + m_2)/2),$$

决策面方程为

$$g(\mathbf{x}) = 0,$$

其中 $\mathbf{x}$ 为向量。

当 $g(\mathbf{x}) > 0$ 时，决策为“1”类，否则为“2”类。

2.4 实验结论与分析

图 2.1 展示了训练集与决策面的可视化图，图 2.2 展示了测试集与决策面可视化图。进而可以计算出模型的准确率 (accuracy)，精确率 (precision)，召回率 (recall) 和 F1 得分的值分别为 99.25%，99.00%，99.50%，99.25%（保留两位小数）。说明模型的预测性能表现优异。

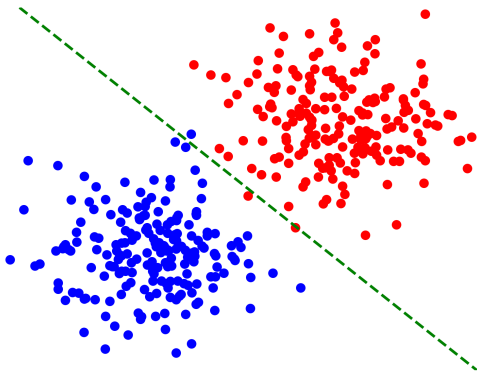


图 2.1: 训练集与决策面

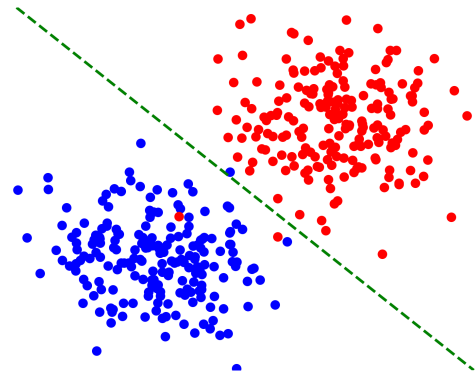


图 2.2: 测试集与决策面

2.5 实验代码

```

1 import os
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import matplotlib as mpl
6
7 mpl.rcParams["font.sans-serif"] = ["SimHei"] # 中文
8 plt.rcParams["axes.unicode_minus"] = False # 负号
9 os.chdir(os.path.dirname(os.path.abspath(__file__))) # 更改
    当前工作目录为脚本所在目录
10
11 train_path = "data/train.txt"
12 test_path = "data/test.txt"
13 output_dir = "output"
14
15
16 def train(data):
17     """学习垂直平分线分类器参数，返回类别标签、中点与法向量。"""
18     means = data.groupby("Label").mean()
19     labels = sorted(means.index.tolist()) # 动态取类别，不假设
        是 1/2
20     m_1, m_2 = means.loc[labels[0]], means.loc[labels[1]]
21     midpoint = (m_1 + m_2) / 2 # 垂直平分线经过的中点
22     normal = m_1 - m_2 # 两簇中心差向量 = 决策线的法向量

```

```
23     return labels, midpoint, normal
24
25
26 def predict(data, labels, midpoint, normal):
27     """用中点 + 法向量判定每个样本落在决策线的哪一侧。"""
28     data["Predict"] = np.where(
29         (data.X - midpoint.X) * normal.X + (data.Y -
30         midpoint.Y) * normal.Y > 0,
31         labels[0],
32         labels[1],
33     )
34     data.to_csv(os.path.join(output_dir, "test_predict.csv")
35     ))
36     return data
37
38
39 def plot(data, labels, midpoint, normal, filename):
40     plt.axline(
41         midpoint,
42         slope=-normal.X / normal.Y,
43         color="green",
44         linewidth=2,
45         linestyle="--",
46         label="Perpendicular Bisector Classifier",
47     )
48     color_map = {labels[0]: "r", labels[1]: "b"}
49     for _, i in data.iterrows():
50         x, y, label = i.iloc[0], i.iloc[1], i.iloc[2]
51         plt.scatter(x, y, color=color_map[label])
52     plt.xlim(-8, 8)
53     plt.ylim(-8, 8)
54     plt.axis("off")
55     plt.savefig(filename, dpi=300, bbox_inches="tight")
56     plt.close()
57
58 def evaluate(data, positive_label):
```

```
58     """计算分类指标, 防止分母为 0 时崩溃。"""
59     TP = TN = FP = FN = 0
60     for _, i in data.iterrows():
61         actual, pred = i.iloc[2], i.iloc[3]
62         if actual == pred:
63             if actual == positive_label:
64                 TP += 1
65             else:
66                 TN += 1
67         else:
68             if actual == positive_label:
69                 FP += 1
70             else:
71                 FN += 1
72     total = TP + TN + FP + FN
73     accuracy = (TP + TN) / total if total else 0.0
74     precision = TP / (TP + FP) if (TP + FP) else 0.0
75     recall = TP / (TP + FN) if (TP + FN) else 0.0
76     f1 = 2 * precision * recall / (precision + recall) if (
77         precision + recall) else 0.0
78     print(f"accuracy:  {accuracy:.4f}")
79     print(f"precision: {precision:.4f}")
80     print(f"recall:    {recall:.4f}")
81     print(f"F1 score:  {f1:.4f}")
82
83 def main():
84     os.makedirs(output_dir, exist_ok=True)
85     train_data = pd.read_csv(train_path, index_col=0)
86     test_data = pd.read_csv(test_path, index_col=0)
87     labels, midpoint, normal = train(train_data)
88     # 绘制训练集与决策界
89     plot(train_data, labels, midpoint, normal, os.path.join(
90         output_dir, "train.png"))
91     # 绘制测试集与决策界
92     plot(test_data, labels, midpoint, normal, os.path.join(
93         output_dir, "test.png"))
```

```
92     test_data = predict(test_data, labels, midpoint, normal
93                          )
94     # 评估测试结果
95     evaluate(test_data, labels[0])
96
97 if __name__ == "__main__":
98     main()
```

第三次上机实验 (使用朴素贝叶斯方法进行预测)

3.1 实验要求

- 给定一定时间内的犯罪数据集, 使用朴素贝叶斯方法进行犯罪类型预测;
- 使用 Python 编程实现。

3.2 数据分析与处理

原始数据包括 Dates, Category, Descript, DayOfWeek, PdDistrict。其中 Category 是我们要尽心预测的对象, 我们对其进行编码, 然后我们对 Dates, DayOfWeek, PdDistrict 进行 one-hot 编码, 利用 TD-IDF 特征从 Descript 中提取关键词并进行编码。

3.3 实验步骤与原理

给定特征 $\mathbf{x} = (x_1, x_2, \dots, x_n)$ 和类别 $\mathbf{v} = (v_1, v_2, \dots, v_m)$ 。在分类任务中, 我们要找到使得 $P(v|\mathbf{x})$ 最大的 v 。利用贝叶斯公式

$$P(v|\mathbf{x}) = \frac{P(\mathbf{x}|v)P(v)}{P(\mathbf{x})},$$

又 $P(\mathbf{x})$ 为定值, 则我们比较 $P(\mathbf{x}|v)P(v)$ 即可。

在朴素贝叶斯中, 我们假设特征 $x_1, x_2, \dots, x_n$ 之间是相互独立的, 即

$$P(\mathbf{x}|v) = \prod_{i=1}^n P(x_i|v).$$

则使得 $P(v|\mathbf{x})$ 最大的

$$v = \arg \max_{v_j} P(v_j) \prod_{i=1}^n P(x_i|v_j)$$

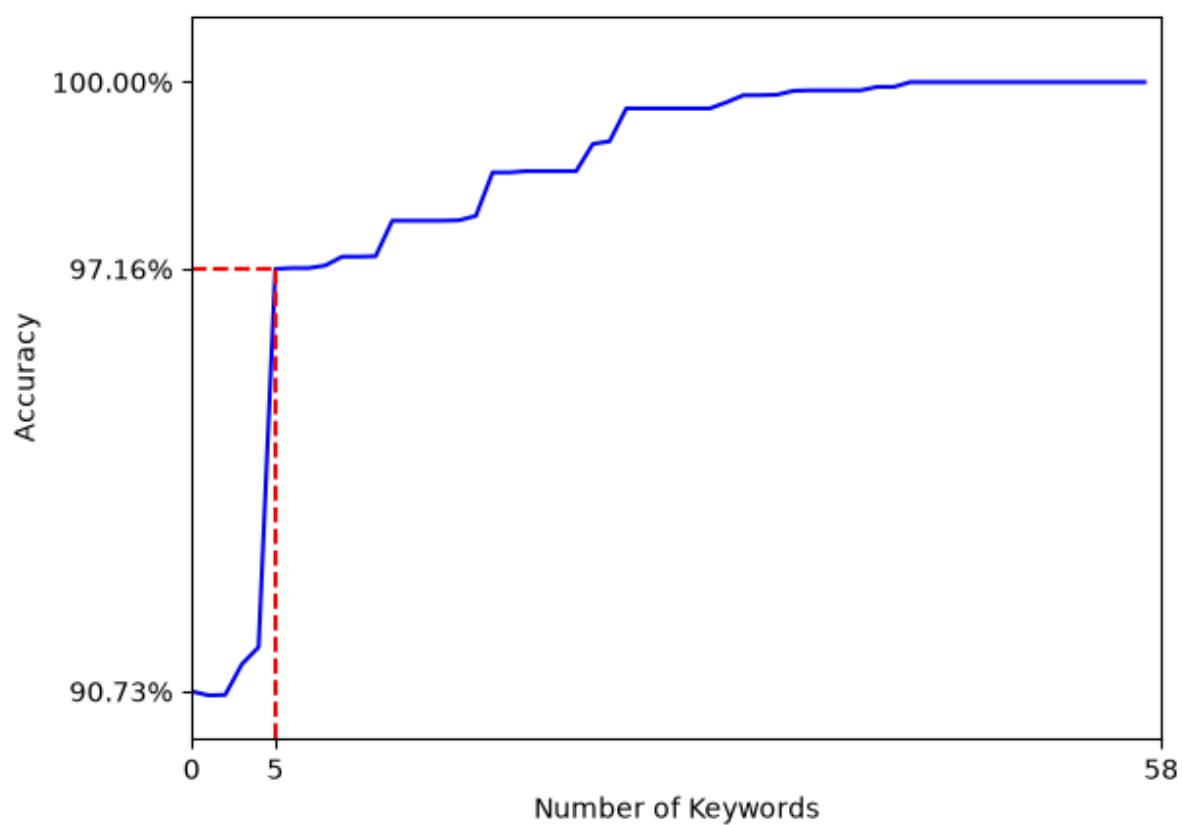


图 3.1: 测试集上的准确率与关键词个数关系图

3.4 实验结论与分析

根据 TF-IDF 值对每一个 Descript 选取一个关键词，选取出现频率最高的 k 个关键词进行模型拟合，在测试集上的准确率如图 3.1 所示。为了防止模型过拟合并保证其泛化能力，选取频率最高的前 5 个关键词。

考虑测试集和训练集 Category 中 DRUG/NARCOTIC 类数据分别占据 90.79% 和 89.81%，为不均衡数据，本实验模型的分类正确率达到了 97.16%，效果较好。

3.5 实验代码

```

1 import os
2 import pandas as pd
3 from sklearn import preprocessing
4 from sklearn.naive_bayes import BernoulliNB
5 from sklearn.metrics import accuracy_score
6 from sklearn.feature_extraction.text import TfidfVectorizer
7 import matplotlib.pyplot as plt
8
9 os.chdir(os.path.dirname(os.path.abspath(__file__))) # 更改
    当前工作目录为脚本所在目录
10
11 train_path = "train.csv"
12 test_path = "test.csv"
13
14
15 def keyword_extract(path="train.csv"):
16     df = pd.read_csv(path)
17     vectorizer = TfidfVectorizer(stop_words="english",
18                                 token_pattern=r"(?u)\b\w+\b")
19     tfidf = pd.DataFrame(
20         vectorizer.fit_transform(df["Descript"]).toarray(),
21         columns=vectorizer.get_feature_names_out(),
22     )
23     keywords = tfidf.idxmax(axis=1)
24     keywords = keywords.value_counts(normalize=True).index.
    str.upper().tolist()

```

```
24     return keywords
25
26
27 def build_encoder(df):
28     """在训练集上学习类别编码，返回共享的标签编码器与 one-hot 规范列
29     名。"""
30     encoder = preprocessing.LabelEncoder()
31     encoder.fit(df["Category"])
32
33     hour_cols = pd.get_dummies(pd.to_datetime(df["Dates"]).
34                               dt.hour).columns
35     day_cols = pd.get_dummies(df["DayOfWeek"]).columns
36     district_cols = pd.get_dummies(df["PdDistrict"]).
37                     columns
38     return encoder, hour_cols, day_cols, district_cols
39
40 def Pre_Process(df, keywords, encoder, hour_cols, day_cols,
41                 district_cols):
42     # 标签编码：用训练集拟合好的编码器 transform (不再重新 fit, 保证映射
43     # 一致)
44     crime_type_encode = encoder.transform(df["Category"])
45
46     # one-hot 编码后按训练集列结构对齐 (test 缺失的类别补 0, 多余的丢弃)
47     hour = pd.get_dummies(pd.to_datetime(df["Dates"]).dt.
48                           hour).reindex(
49         columns=hour_cols, fill_value=0
50     )
51     day = pd.get_dummies(df["DayOfWeek"]).reindex(
52         columns=day_cols, fill_value=0
53     )
54     police_district = pd.get_dummies(df["PdDistrict"]).
55         reindex(
56             columns=district_cols, fill_value=0
57         )
58
59     # 利用 TF-IDF 特征进行编码
```

```
54     matrix = pd.DataFrame(0, index=df.index, columns=
        keywords)
55     for keyword in keywords:
56         matrix[keyword] = df["Descript"].apply(
57             lambda x: True if keyword in x else False
58         )
59
60     # 将特征合并
61     data = pd.concat([hour, day, police_district, matrix],
        axis=1)
62     data["Crime type"] = crime_type_encode
63
64     # Feature names are only supported if all input features
        have string names
65     data.columns = data.columns.astype(str)
66     return data
67
68
69 train_df = pd.read_csv(train_path)
70 test_df = pd.read_csv(test_path)
71 keywords = keyword_extract()
72 encoder, hour_cols, day_cols, district_cols = build_encoder
        (train_df)
73
74 acc = []
75 for k in range(0, len(keywords) + 1):
76     train = Pre_Process(
77         train_df, keywords[:k], encoder, hour_cols,
78         day_cols, district_cols
79     )
80     test = Pre_Process(
81         test_df, keywords[:k], encoder, hour_cols, day_cols
82         , district_cols
83     )
84
85     # 训练模型
86     model = BernoulliNB()
```

```
84     model.fit(train.drop("Crime type", axis=1), train["
        Crime type"])
85
86     # 预测结果
87     pred = model.predict(test.drop("Crime type", axis=1))
88     acc.append(
89         accuracy_score(test["Crime type"], pred),
90     )
91
92     # plot
93     plt.plot(range(0, len(keywords) + 1), acc, color="b")
94     plt.xlabel("Number of Keywords")
95     plt.ylabel("Accuracy")
96     points = [(0, acc[0]), (5, acc[5]), (len(acc), acc[len(acc)
        - 1])]
97     xticks = [x for x, _ in points]
98     yticks = [y for _, y in points]
99     plt.xticks(xticks, [f"{x}" for x in xticks])
100    plt.yticks(yticks, [f"{y * 100:.2f}%" for y in yticks])
101    plt.plot([points[1][0], points[1][0]], [0.90, points
        [1][1]], color="r", linestyle="--")
102    plt.plot([0.00, points[1][0]], [points[1][1], points
        [1][1]], color="r", linestyle="--")
103    plt.xlim(0, len(keywords) + 1)
104    plt.ylim(0.90, 1.01)
105    # plt.show()
106    plt.savefig("accuracy.png", bbox_inches="tight")
```

第四次上机实验 (使用决策树进行预测)

4.1 实验要求

- 给定一定时间内的犯罪数据集，使用决策树方法进行犯罪类型预测；
- 使用 Python 编程实现。

4.2 数据分析与处理

同 3.2 节。

4.3 实验步骤与原理

本试验采用 CART 算法，其不纯度 (Gini 系数) 定义如下：

$$\text{Gini} = 1 - \sum_{i=1}^N p_i^2,$$

其中 p_i 是每个类 i 的概率， N 是类别总数。信息增益定义为依某个特征进行分类前后的不纯度之差，选择使得信息增益最高的特征。

4.4 实验结论与分析

根据 TF-IDF 值对每一个 Descript 选取一个关键词，选取出现频率最高的 k 个关键词进行模型拟合，在测试集上的准确率如图 4.1 所示。可以看出决策树的训练与大部分频率较高的关键词并无较大帮助，部分频率较低的关键词反而可以很好地减少决策树的深度。我们选取所有关键词进行模型训练，最终决策树如图 4.2，训练集上的准确率达到 100%。

4.5 实验代码

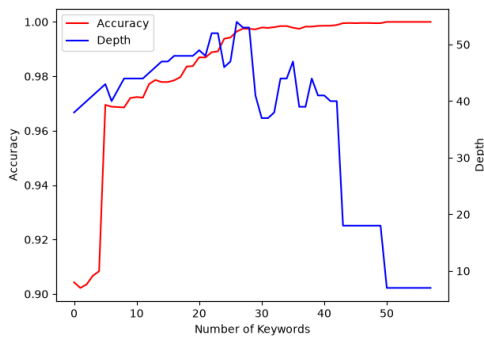


图 4.1: 决策树深度和测试集上的准确率和与关键词个数关系图

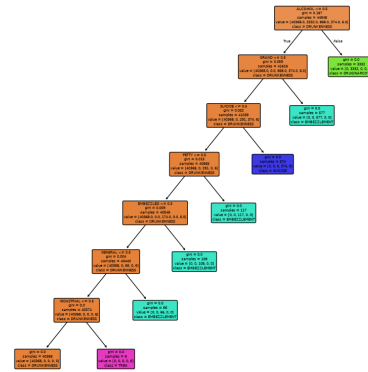


图 4.2: 决策树可视化图 (Depth=7)

```

1  import os
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from sklearn import preprocessing
5  from sklearn.metrics import accuracy_score
6  from sklearn.feature_extraction.text import TfidfVectorizer
7  from sklearn.tree import DecisionTreeClassifier, plot_tree
8
9
10 os.chdir(os.path.dirname(os.path.abspath(__file__))) # 更改
    当前工作目录为脚本所在目录
11
12 train_path = "train.csv"
13 test_path = "test.csv"
14
15
16 def keyword_extract(path="train.csv"):
17     df = pd.read_csv(path)
18     vectorizer = TfidfVectorizer(stop_words="english",
19                                 token_pattern=r"(?u)\b\w+\b")
19     tfidf = pd.DataFrame(
20         vectorizer.fit_transform(df["Descript"]).toarray(),
21         columns=vectorizer.get_feature_names_out(),
22     )

```

```
23     keywords = tfidf.idxmax(axis=1)
24     keywords = keywords.value_counts(normalize=True).index.
        str.upper().tolist()
25     return keywords
26
27
28 def build_encoder(df):
29     """在训练集上学习类别编码，返回共享的标签编码器与 one-hot 规范列
        名。"""
30     encoder = preprocessing.LabelEncoder()
31     encoder.fit(df["Category"])
32
33     hour_cols = pd.get_dummies(pd.to_datetime(df["Dates"]).
        dt.hour).columns
34     day_cols = pd.get_dummies(df["DayOfWeek"]).columns
35     district_cols = pd.get_dummies(df["PdDistrict"]).
        columns
36     return encoder, hour_cols, day_cols, district_cols
37
38
39 def Pre_Process(df, keywords, encoder, hour_cols, day_cols,
        district_cols):
40     # 标签编码：用训练集拟合好的编码器 transform (不再重新 fit, 保证映射
        一致)
41     crime_type_encode = encoder.transform(df["Category"])
42
43     # one-hot 编码后按训练集列结构对齐 (test 缺失的类别补 0, 多余的丢弃)
44     hour = pd.get_dummies(pd.to_datetime(df["Dates"]).dt.
        hour).reindex(
45         columns=hour_cols, fill_value=0
46     )
47     day = pd.get_dummies(df["DayOfWeek"]).reindex(
48         columns=day_cols, fill_value=0
49     )
50     police_district = pd.get_dummies(df["PdDistrict"]).
        reindex(
51         columns=district_cols, fill_value=0
```

```
52     )
53
54     # 利用 TF-IDF 特征进行编码
55     matrix = pd.DataFrame(0, index=df.index, columns=
        keywords)
56     for keyword in keywords:
57         matrix[keyword] = df["Descript"].apply(
58             lambda x: True if keyword in x else False
59         )
60
61     # 将特征合并
62     data = pd.concat([hour, day, police_district, matrix],
        axis=1)
63     data["Crime type"] = crime_type_encode
64
65     # Feature names are only supported if all input features
        have string names
66     data.columns = data.columns.astype(str)
67     return data
68
69
70 train_df = pd.read_csv(train_path)
71 test_df = pd.read_csv(test_path)
72 keywords = keyword_extract()
73 encoder, hour_cols, day_cols, district_cols = build_encoder
        (train_df)
74
75 acc = []
76 depth = []
77 for k in range(0, len(keywords) + 1):
78     train = Pre_Process(
79         train_df, keywords[:k], encoder, hour_cols,
80         day_cols, district_cols
81     )
82     test = Pre_Process(
83         test_df, keywords[:k], encoder, hour_cols, day_cols
84         , district_cols
```

```
83     )
84     # 训练模型
85     model = DecisionTreeClassifier()
86     model.fit(train.drop("Crime type", axis=1), train["
        Crime type"])
87     # 预测结果
88     y_pred = model.predict(test.drop("Crime type", axis=1))
89     acc.append(accuracy_score(test["Crime type"], y_pred))
90     depth.append(model.get_depth())
91
92     # plot
93     fig, ax1 = plt.subplots()
94     ax1.set_xlabel("Number of Keywords")
95     ax1.set_ylabel("Accuracy")
96     ax1.plot(range(0, len(keywords) + 1), acc, color="r", label
        ="Accuracy")
97     ax1.tick_params(axis="y")
98     # 创建第二个坐标轴共享x轴
99     ax2 = ax1.twinx()
100    ax2.set_ylabel("Depth")
101    ax2.plot(range(0, len(keywords) + 1), depth, color="b",
        label="Depth")
102    ax2.tick_params(axis="y")
103    # 显示图例
104    lines, labels = ax1.get_legend_handles_labels()
105    lines2, labels2 = ax2.get_legend_handles_labels()
106    ax1.legend(lines + lines2, labels + labels2, loc="upper
        left")
107    # plt.show()
108    plt.savefig("accuracy+depth.png")
109    # 绘制决策树
110
111    plt.figure(figsize=(10, 10))
112    plot_tree(
113        model,
114        filled=True,
115        rounded=True,
```

```
116     class_names=list(train_df["Category"].unique()),  
117     feature_names=list(train.drop("Crime type", axis=1).  
        columns),  
118 )  
119 plt.savefig("decision_tree.png")
```

第五次上机实验 (信用卡欺诈预测)

5.1 实验要求

- 给定信用卡欺诈数据集, 判断是否存在欺诈
- 请使用 BP 神经网络或基于 delta 准则的感知机实现

5.2 数据分析与处理

测试集和训练集均已划分好, 包括进行 PCA 处理后的 30 个特征, 对其进行归一化处理。

5.3 实验步骤与原理

本实验采用 BP 神经网络, Input layer 是一个 30 维的数据, Hidden layer 设定为 30 维, Output layer 为 2 维标签数据, 激活函数为 sigmoid 函数。

梯度下降法则为:

$$\Delta w = -\eta \nabla E(w)$$

其中 w 为模型参数, η 为学习率, ∇ 为梯度算子, E 为总偏差。

下面我们利用反向传播, 来计算 $\nabla E(w)$ 。总偏差定义为

$$E = \frac{1}{2} \sum_c \sum_j (y_{j,c} - d_{j,c})^2 \quad (5.1)$$

其中 c 是案例 (输入-输出对) 的索引, j 是输出单元的索引, y 是输出单元的实际值而 d 是目标值. 我们的 * 目标 * 变为寻找 $\operatorname{argmin}_w E$. 不失一般性, 我们考虑单元 j 的输入 x_j 与输出 y_j , 总输入 x_j 定义为下层单元输出值的线性函数:

$$x_j = \sum_i y_i w_{ji} \quad (5.2)$$

其中 y_i 为第 i 个单元的输入, w_{ji} 表示连接第 i 个单元到第 j 个单元的参数. 每个单元都有其输出值, 通过非线性函数 Sigmoid 函数定义

$$y_j = \frac{1}{1 + e^{-x_j}} \quad (5.3)$$

按照反向传播的定义, 我们对 E 关于 y 求偏导. 固定 5.1 中的 c , 对单元 j 的输出 y_j 求偏导

$$\frac{\partial E}{\partial y_j} = y_j - d_j \quad (5.4)$$

再对 x_j 求偏导

$$\frac{\partial y_j}{\partial x_j} = y_j(1 - y_j) \quad (5.5)$$

对 5.4 与 5.5 应用链式法则

$$\frac{\partial E}{\partial x_j} = \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_j} = (y_j - d_j) \cdot y_j(1 - y_j) \quad (5.6)$$

再次利用链式法则与 5.2 对 w_{ji} 求偏导

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial x_j} \cdot \frac{\partial x_j}{\partial w_{ji}} = \frac{\partial E}{\partial x_j} \cdot y_i \quad (5.7)$$

5.4 实验结论与分析

本实验的准确率为 99.20%, AUC 值为 0.9038 (见图 5.1), 可以认为模型分类效果优秀。

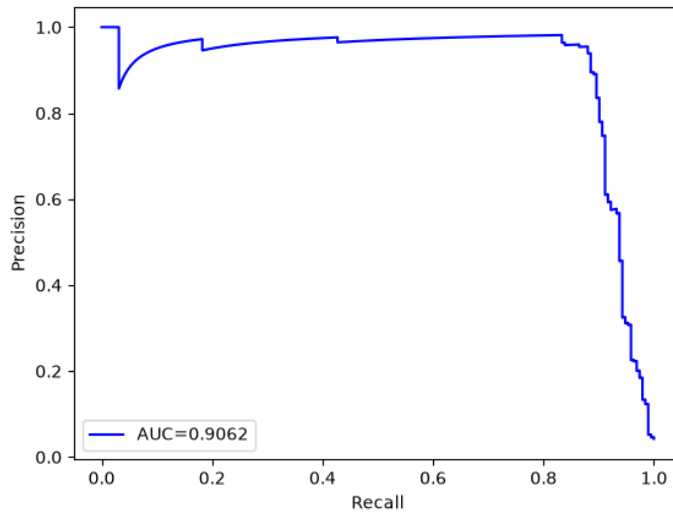


图 5.1: Recall-Precision 曲线图

5.5 实验代码

```
1 import os
2 import pandas as pd
3 import torch
4 import torch.nn as nn
5 import matplotlib.pyplot as plt
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import precision_recall_curve, auc
8
9 os.chdir(os.path.dirname(os.path.abspath(__file__))) # 更改
    当前工作目录为脚本所在目录
10 train_path = "train.csv"
11 test_path = "test.csv"
12
13
14 def Pre_process(path):
15     """读取数据，返回原始特征 X 和标签 y (不做归一化) 。"""
16     data = pd.read_csv(path, index_col=0)
17     X = data.drop("Class", axis=1).values
18     y = data["Class"].values
19     return X, y
20
21
22 class BpNet(nn.Module):
23     def __init__(self, input_dim, hidden_dim, output_dim):
24         super(BpNet, self).__init__()
25         self.fc1 = nn.Linear(input_dim, hidden_dim)
26         self.fc2 = nn.Linear(hidden_dim, output_dim)
27         self.sigmoid = nn.Sigmoid()
28
29     def forward(self, x):
30         x = self.sigmoid(self.fc1(x))
31         x = self.fc2(x)
32         return x
33
34
35 scaler = StandardScaler()
36 X_train, y_train = Pre_process(train_path)
```

```
37 X_test, y_test = Pre_process(test_path)
38
39 # 归一化: 只在训练集上 fit (学习均值/方差), 测试集仅 transform (沿用训练
    集统计量)
40 # 避免测试集的统计量泄漏进评估, 保证指标反映真实泛化能力
41 X_train = scaler.fit_transform(X_train)
42 X_test = scaler.transform(X_test)
43
44 X_train = torch.from_numpy(X_train).float()
45 y_train = torch.from_numpy(y_train).long()
46 X_test = torch.from_numpy(X_test).float()
47 y_test = torch.from_numpy(y_test).long()
48
49 seed = 233
50 epochs = 100
51 torch.manual_seed(seed)
52 model = BpNet(input_dim=X_train.shape[1], hidden_dim=30,
    output_dim=2)
53 criterion = nn.CrossEntropyLoss()
54 optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
55
56 for epoch in range(epochs):
57     model.train()
58     optimizer.zero_grad()
59     output = model(X_train)
60     loss = criterion(output, y_train)
61     loss.backward()
62     optimizer.step()
63     if epoch % 10 == 0:
64         print("Epoch: {}/{}, Loss: {:.4f}".format(epoch +
            1, epochs, loss.item()))
65
66 model.eval()
67 with torch.no_grad():
68     output = model(X_test)
69     pred = output.argmax(dim=1, keepdim=True)
70     correct = pred.eq(y_test.view_as(pred)).sum().item()
```

```
71     print("Accuracy: {:.2f}%".format(correct / len(y_test)
72           * 100))
73
74 # AUC
75 y_score = output[:, 1].numpy()
76 precision, recall, _ = precision_recall_curve(y_test.numpy()
77       (), y_score)
78 area = auc(recall, precision)
79 print("AUC: {:.4f}".format(area))
80
81 # plot auc
82 plt.plot(recall, precision, label="AUC={:.4f}".format(area)
83       , color="b")
84 plt.xlabel("Recall")
85 plt.ylabel("Precision")
86 plt.legend(loc="best")
87 # plt.show()
88 plt.savefig("auc.png")
```